

Université Mohammed VI Polytechnique

College of Computing

TP6

Report

Module: Foundations of Parallel Computing

Instructor: Imad Kissami

Prepared by:

Malak Kably

Academic Year: 2025–2026

Contents

1	Exercise 1: Matrix Transposition with MPI Derived Types	2
1.1	Implementation Details	2
1.1.1	Step 1: Define and Initialize Matrix	2
1.1.2	Step 2: Display Matrix Before Sending	2
1.1.3	Step 3: Declare Transposed Matrix on Process 1	2
1.1.4	Step 4: Build Derived Datatype	3
1.1.5	Step 5: Commit and Use in MPI_Recv	3
1.1.6	Step 6: Send Original Matrix	4
1.1.7	Step 7: Display Transposed Matrix	4
1.2	Results	4
2	Exercise 2: Distributed Gradient Descent with MPI Derived Types	5
2.1	Problem Formulation	5
2.2	Implementation Details	6
2.2.1	Step 1: Define Derived Type for Sample Struct	6
2.2.2	Step 2: Generate Dataset on Process 0	6
2.2.3	Step 3: Scatter Dataset Using MPI_Scatterv	7
2.2.4	Step 4: Local Gradient and Loss Computation	8
2.2.5	Step 5: Aggregate Gradients and Losses	8
2.2.6	Step 6: Update Weight Vector	9
2.2.7	Step 7: Print Progress Every 10 Epochs	9
2.2.8	Step 8: Early Stopping	9
2.2.9	Step 9: Performance Analysis	10
2.3	Experimental Setup	10
2.4	Data Collection	10
2.5	Performance Metrics	11
2.5.1	Speedup	11
2.5.2	Efficiency	11
2.5.3	Amdahl's Law	11
2.6	Results and Analysis	12

1 Exercise 1: Matrix Transposition with MPI Derived Types

1.1 Implementation Details

1.1.1 Step 1: Define and Initialize Matrix

On process 0, we define a 4×5 matrix and fill it with values from 1 to 20:

```
1 #define ROWS 4
2 #define COLS 5
3
4 int matrix[ROWS][COLS];
5
6 if (rank == 0) {
7     int value = 1;
8     for (int i = 0; i < ROWS; i++) {
9         for (int j = 0; j < COLS; j++) {
10             matrix[i][j] = value++;
11         }
12     }
13     print_matrix(matrix, ROWS, COLS, rank, "Original Matrix a");
14 }
```

Listing 1: Matrix initialization on process 0

1.1.2 Step 2: Display Matrix Before Sending

The original matrix is displayed on process 0 to verify initialization:

```
1 void print_matrix(int matrix[][COLS], int rows, int cols,
2                  int rank, const char* title) {
3     printf("\nProcess %d - %s:\n", rank, title);
4     for (int i = 0; i < rows; i++) {
5         for (int j = 0; j < cols; j++) {
6             printf("%3d ", matrix[i][j]);
7         }
8         printf("\n");
9     }
10 }
```

Listing 2: Display function

1.1.3 Step 3: Declare Transposed Matrix on Process 1

Process 1 declares a matrix with dimensions 5×4 to hold the transposed result:

```
1 int transposed[COLS][ROWS]; // 5x4 matrix for transposed data
```

Listing 3: Transposed matrix declaration

1.1.4 Step 4: Build Derived Datatype

We create a custom MPI datatype to extract columns from the original matrix. The `MPI_Type_vector` function allows us to define non-contiguous data patterns:

```
1 MPI_Datatype column_type;
2 MPI_Type_vector(
3     ROWS,           // count: number of elements in a column
4     1,              // blocklength: one element per block
5     COLS,           // stride: jump to next row (skip COLS elements)
6     MPI_INT,        // oldtype: base datatype
7     &column_type    // newtype: output derived type
8 );
9 MPI_Type_commit(&column_type);
```

Listing 4: Creating column extraction datatype

The datatype is then resized to ensure correct extent for sending multiple columns:

```
1 MPI_Datatype column_type_resized;
2 MPI_Type_create_resized(
3     column_type,      // input type
4     0,               // lower bound
5     sizeof(int),      // extent: move one integer per column
6     &column_type_resized // output type
7 );
8 MPI_Type_commit(&column_type_resized);
```

Listing 5: Resizing the derived type

1.1.5 Step 5: Commit and Use in MPI_Recv

Process 0 sends the entire matrix using the custom derived type, while process 1 receives it as a contiguous block:

```
1 if (rank == 0) {
2     MPI_Send(
3         &matrix[0][0],           // buffer: start of matrix
4         COLS,                    // count: send all columns
5         column_type_resized,      // datatype: custom column type
6         1,                       // dest: process 1
7         0,                       // tag
8         MPI_COMM_WORLD
9     );
10 }
```

Listing 6: Send operation on process 0

```
1 if (rank == 1) {
2     MPI_Recv(
3         &transposed[0][0], // buffer: transposed matrix
```

```

4      ROWS * COLS,          // count: total elements
5      MPI_INT,              // datatype: regular integers
6      0,                    // source: process 0
7      0,                    // tag
8      MPI_COMM_WORLD,
9      MPI_STATUS_IGNORE
10   );
11   print_transposed_matrix(transposed, COLS, ROWS, rank,
12                           "Matrix transposed at");
13 }

```

Listing 7: Receive operation on process 1

1.1.6 Step 6: Send Original Matrix

The matrix is sent using `MPI_Send` with the custom derived type, which automatically extracts columns and sends them sequentially.

1.1.7 Step 7: Display Transposed Matrix

Process 1 displays the received transposed matrix to verify correct data transfer:

```

1 void print_transposed_matrix(int matrix[][ROWS], int rows, int cols,
2                             int rank, const char* title) {
3     printf("\nProcess %d - %s:\n", rank, title);
4     for (int i = 0; i < rows; i++) {
5         for (int j = 0; j < cols; j++) {
6             printf("%3d ", matrix[i][j]);
7         }
8         printf("\n");
9     }
10 }

```

Listing 8: Display transposed matrix

1.2 Results

The full implementation can be found in `code.c`. The program successfully transposes the 4×5 matrix into a 5×4 matrix using a single `MPI_Recv` call, as shown in the output below:

```

malak@LAPTOP-PQ0A3CAA:~/ParallelAndDistributedProgramming/TP6/ex1$ nano code.c
malak@LAPTOP-PQ0A3CAA:~/ParallelAndDistributedProgramming/TP6/ex1$ mpicc -o code code.c -Wall
malak@LAPTOP-PQ0A3CAA:~/ParallelAndDistributedProgramming/TP6/ex1$ mpirun -np 2 ./code

Process 1 - Matrix transposed at:
 1  6 11 16
 2  7 12 17
 3  8 13 18
 4  9 14 19
 5 10 15 20

Process 0 - Original Matrix a:
 1  2  3  4  5
 6  7  8  9 10
11 12 13 14 15
16 17 18 19 20

```

Figure 1: Program output showing original matrix on process 0 and transposed matrix on process 1

The output demonstrates:

- **Process 0:** Displays the original 4×5 matrix with values 1-20 in row-major order
- **Process 1:** Displays the transposed 5×4 matrix where rows and columns are swapped

2 Exercise 2: Distributed Gradient Descent with MPI Derived Types

2.1 Problem Formulation

We aim to train a linear regression model:

$$\hat{y} = \sum_{j=0}^{N_{features}-1} w_j \cdot x_j \quad (1)$$

where w_j are the weights to be learned, and x_j are the features. The training objective is to minimize the Mean Squared Error (MSE):

$$L = \frac{1}{N} \sum_{i=1}^N (\hat{y}_i - y_i)^2 \quad (2)$$

using gradient descent with the update rule:

$$w_j \leftarrow w_j - \alpha \cdot \frac{\partial L}{\partial w_j} \quad (3)$$

where α is the learning rate and:

$$\frac{\partial L}{\partial w_j} = \frac{2}{N} \sum_{i=1}^N (\hat{y}_i - y_i) \cdot x_{i,j} \quad (4)$$

2.2 Implementation Details

2.2.1 Step 1: Define Derived Type for Sample Struct

We define a custom MPI datatype for the `Sample` structure, which contains 5 features and one target value:

```
1 #define N_FEATURES 5
2
3 typedef struct {
4     double x[N_FEATURES]; // Feature vector
5     double y;             // Target value
6 } Sample;
```

Listing 9: Sample structure definition

To enable efficient communication of this struct, we create a derived MPI datatype using `MPI_Type_create_struct`:

```
1 MPI_Datatype create_sample_type() {
2     MPI_Datatype sample_type;
3     int blocklengths[2] = {N_FEATURES, 1}; // 5 doubles + 1 double
4     MPI_Aint displacements[2];
5     MPI_Datatype types[2] = {MPI_DOUBLE, MPI_DOUBLE};
6
7     // Calculate memory offsets
8     Sample dummy_sample;
9     (void)dummy_sample; // Avoid uninitialized warning
10    MPI_Aint base_address;
11    MPI_Get_address(&dummy_sample, &base_address);
12    MPI_Get_address(&dummy_sample.x, &displacements[0]);
13    MPI_Get_address(&dummy_sample.y, &displacements[1]);
14
15    // Make displacements relative to base
16    displacements[0] = MPI_Aint_diff(displacements[0], base_address);
17    displacements[1] = MPI_Aint_diff(displacements[1], base_address);
18
19    // Create and commit the derived type
20    MPI_Type_create_struct(2, blocklengths, displacements,
21                          types, &sample_type);
22    MPI_Type_commit(&sample_type);
23
24    return sample_type;
25 }
```

Listing 10: Creating MPI derived type for Sample struct

2.2.2 Step 2: Generate Dataset on Process 0

Process 0 generates the full synthetic dataset using predefined true weights to validate the learning algorithm:

```

1 Sample* generate_data(int n_samples) {
2     Sample *data = (Sample*)malloc(n_samples * sizeof(Sample));
3     double true_weights[N_FEATURES] = {2.5, -1.3, 0.8, 3.2, -0.5};
4
5     srand(42); // Fixed seed for reproducibility
6     for (int i = 0; i < n_samples; i++) {
7         // Generate random features in range [-5, 5]
8         for (int j = 0; j < N_FEATURES; j++) {
9             data[i].x[j] = ((double)rand() / RAND_MAX) * 10.0 - 5.0;
10        }
11
12        // Generate target using true weights + small noise
13        data[i].y = 0.0;
14        for (int j = 0; j < N_FEATURES; j++) {
15            data[i].y += true_weights[j] * data[i].x[j];
16        }
17        data[i].y += ((double)rand() / RAND_MAX) * 0.5 - 0.25;
18    }
19    return data;
20 }

```

Listing 11: Dataset generation with known ground truth

The dataset is generated only on process 0:

```

1 Sample *dataset = NULL;
2 if (rank == 0) {
3     dataset = generate_data(n_samples);
4 }

```

Listing 12: Dataset initialization on root process

2.2.3 Step 3: Scatter Dataset Using MPI_Scatterv

We distribute the dataset across all processes using `MPI_Scatterv` with the custom derived type. This handles uneven distribution when the number of samples is not perfectly divisible:

```

1 int samples_per_proc = n_samples / num_procs;
2 int extra_samples = n_samples % num_procs;
3 int local_n_samples = samples_per_proc + (rank < extra_samples ? 1 : 0);
4
5 // Prepare scatter parameters on process 0
6 int *sendcounts = NULL, *displs = NULL;
7 if (rank == 0) {
8     sendcounts = (int*)malloc(num_procs * sizeof(int));
9     displs = (int*)malloc(num_procs * sizeof(int));
10    int offset = 0;
11    for (int i = 0; i < num_procs; i++) {
12        sendcounts[i] = samples_per_proc + (i < extra_samples ? 1 : 0);

```



```

13     displs[i] = offset;
14     offset += sendcounts[i];
15 }
16 }

```

Listing 13: Workload distribution calculation

```

1 Sample *local_dataset = (Sample*)malloc(local_n_samples * sizeof(Sample)
2 );
3 MPI_Scatterv(
4     dataset, sendcounts, displs, sample_type,      // Send buffer
5     local_dataset, local_n_samples, sample_type,    // Receive buffer
6     0, MPI_COMM_WORLD
7 );

```

Listing 14: Scattering dataset with derived type

2.2.4 Step 4: Local Gradient and Loss Computation

Each process computes gradients and loss for its local subset of samples:

```

1 double local_gradients[N_FEATURES] = {0.0};
2 double local_loss = 0.0;
3
4 for (int i = 0; i < local_n_samples; i++) {
5     // Forward pass: compute prediction
6     double prediction = 0.0;
7     for (int j = 0; j < N_FEATURES; j++) {
8         prediction += weights[j] * local_dataset[i].x[j];
9     }
10
11     // Compute error
12     double error = prediction - local_dataset[i].y;
13     local_loss += error * error; // MSE accumulation
14
15     // Backward pass: accumulate gradients
16     for (int j = 0; j < N_FEATURES; j++) {
17         local_gradients[j] += error * local_dataset[i].x[j];
18     }
19 }

```

Listing 15: Local gradient computation

2.2.5 Step 5: Aggregate Gradients and Losses

All processes send their local gradients and losses to process 0 using MPI_Reduce:

```

1 double global_gradients[N_FEATURES];
2 double global_loss;

```

```

3
4 MPI_Reduce(local_gradients, global_gradients, N_FEATURES,
5           MPI_DOUBLE, MPI_SUM, 0, MPI_COMM_WORLD);
6 MPI_Reduce(&local_loss, &global_loss, 1,
7           MPI_DOUBLE, MPI_SUM, 0, MPI_COMM_WORLD);

```

Listing 16: Global reduction of gradients and loss

2.2.6 Step 6: Update Weight Vector

Process 0 updates the global weight vector using the aggregated gradients:

```

1 if (rank == 0) {
2     global_loss = global_loss / n_samples; // Compute MSE
3
4     // Gradient descent update
5     for (int j = 0; j < N_FEATURES; j++) {
6         weights[j] -= LEARNING_RATE * (global_gradients[j] / n_samples);
7     }
8 }

```

Listing 17: Weight update on master process

Before each iteration, the updated weights are broadcasted to all processes:

```

1 MPI_Bcast(weights, N_FEATURES, MPI_DOUBLE, 0, MPI_COMM_WORLD);

```

Listing 18: Broadcasting weights to all processes

2.2.7 Step 7: Print Progress Every 10 Epochs

Process 0 monitors training progress and prints updates periodically:

```

1 if (rank == 0) {
2     if (epoch % 10 == 0) {
3         printf("Epoch %d | Loss (MSE): %.6f | w[0]: %.4f, w[1]: %.4f\n",
4               epoch, global_loss, weights[0], weights[1]);
5     }
6 }

```

Listing 19: Training progress monitoring

2.2.8 Step 8: Early Stopping

Training stops when the global loss falls below a predefined threshold:

```

1 if (rank == 0) {
2     if (global_loss < THRESHOLD) {
3         printf("Early stopping at epoch %d - loss %.5e < %.0e\n",
4               epoch, global_loss, THRESHOLD);
5         converged = 1;

```

```

6     }
7 }
8
9 // Broadcast convergence flag to all processes
10 MPI_Bcast(&converged, 1, MPI_INT, 0, MPI_COMM_WORLD);

```

Listing 20: Convergence check and early stopping

2.2.9 Step 9: Performance Analysis

We measure speedup and efficiency by varying the number of processes and dataset sizes. The complete implementation is available in `code.c`.

2.3 Experimental Setup

We conducted performance tests with the following configurations:

- **Dataset sizes:** 1,000, 5,000, and 10,000 samples
- **Number of processes:** 1, 2, 4 (serial baseline and parallel configurations)
- **Hardware:** Multi-core CPU with MPI support
- **Metrics collected:** Execution time, number of epochs, final loss

2.4 Data Collection

Performance data was collected using automated shell commands that run the program with different configurations and store results in CSV format:

```

malak@LAPTOP-PQ6A3CAA:~/ParallelAndDistributedProgramming/TP6/ex2$ echo "N_SAMPLES,N_PROCESSES,PARALLEL_TIME,EPOCHS,FINAL_LOSS" > results.csv
malak@LAPTOP-PQ6A3CAA:~/ParallelAndDistributedProgramming/TP6/ex2$ mpirun -np 4 ./code 1000 | grep -A1 "^N_SAMPLES,N_PROCESSES" |
tail -1 >> results.csv
malak@LAPTOP-PQ6A3CAA:~/ParallelAndDistributedProgramming/TP6/ex2$ mpirun -np 1 ./code 1000 | grep -A1 "^N_SAMPLES,N_PROCESSES" |
tail -1 >> results.csv
malak@LAPTOP-PQ6A3CAA:~/ParallelAndDistributedProgramming/TP6/ex2$ mpirun -np 2 ./code 1000 | grep -A1 "^N_SAMPLES,N_PROCESSES" |
tail -1 >> results.csv
malak@LAPTOP-PQ6A3CAA:~/ParallelAndDistributedProgramming/TP6/ex2$ mpirun -np 1 ./code 5000 | grep -A1 "^N_SAMPLES,N_PROCESSES" |
tail -1 >> results.csv
malak@LAPTOP-PQ6A3CAA:~/ParallelAndDistributedProgramming/TP6/ex2$ mpirun -np 2 ./code 5000 | grep -A1 "^N_SAMPLES,N_PROCESSES" |
tail -1 >> results.csv
malak@LAPTOP-PQ6A3CAA:~/ParallelAndDistributedProgramming/TP6/ex2$ mpirun -np 4 ./code 5000 | grep -A1 "^N_SAMPLES,N_PROCESSES" |
tail -1 >> results.csv
malak@LAPTOP-PQ6A3CAA:~/ParallelAndDistributedProgramming/TP6/ex2$ mpirun -np 4 ./code 10000 | grep -A1 "^N_SAMPLES,N_PROCESSES" |
tail -1 >> results.csv
malak@LAPTOP-PQ6A3CAA:~/ParallelAndDistributedProgramming/TP6/ex2$ cat results.csv
N_SAMPLES,N_PROCESSES,PARALLEL_TIME,EPOCHS,FINAL_LOSS
1000,4,0.004226,1000,0.021366
1000,1,0.004494,1000,0.021366
1000,2,0.004353,1000,0.021366
5000,1,0.014673,1000,0.021336
5000,2,0.010071,1000,0.021336
5000,4,0.009909,1000,0.021336
malak@LAPTOP-PQ6A3CAA:~/ParallelAndDistributedProgramming/TP6/ex2$

```

Figure 2: Shell commands for automated performance data collection

The results were aggregated in a CSV file (`TP6/ex2/results.csv`) with the following structure. For complete details and raw data, refer to `results.csv` in the project directory.

N_SAMPLES	N_PROCESSES	PARALLEL_TIME	EPOCHS	FINAL_LOSS
1000	1	0.004494	1000	0.021366
1000	2	0.004353	1000	0.021366
1000	4	0.004226	1000	0.021366

2.5 Performance Metrics

2.5.1 Speedup

Speedup measures how much faster the parallel version runs compared to the sequential baseline:

$$S(p) = \frac{T_{\text{sequential}}}{T_{\text{parallel}}(p)} \quad (5)$$

where p is the number of processes, $T_{\text{sequential}}$ is the execution time with 1 process, and $T_{\text{parallel}}(p)$ is the execution time with p processes.

Ideal (linear) speedup occurs when $S(p) = p$, meaning each additional process contributes equally to performance improvement.

2.5.2 Efficiency

Efficiency measures how well the parallel resources are utilized:

$$E(p) = \frac{S(p)}{p} \times 100\% \quad (6)$$

An efficiency of 100% indicates perfect scaling, while values below 100% indicate overhead from communication, synchronization, or load imbalance.

2.5.3 Amdahl's Law

The theoretical maximum speedup is limited by the sequential portion of the algorithm:

$$S_{\text{max}} = \frac{1}{(1 - P) + \frac{P}{N}} \quad (7)$$

where P is the proportion of the program that can be parallelized, and N is the number of processors.

2.6 Results and Analysis

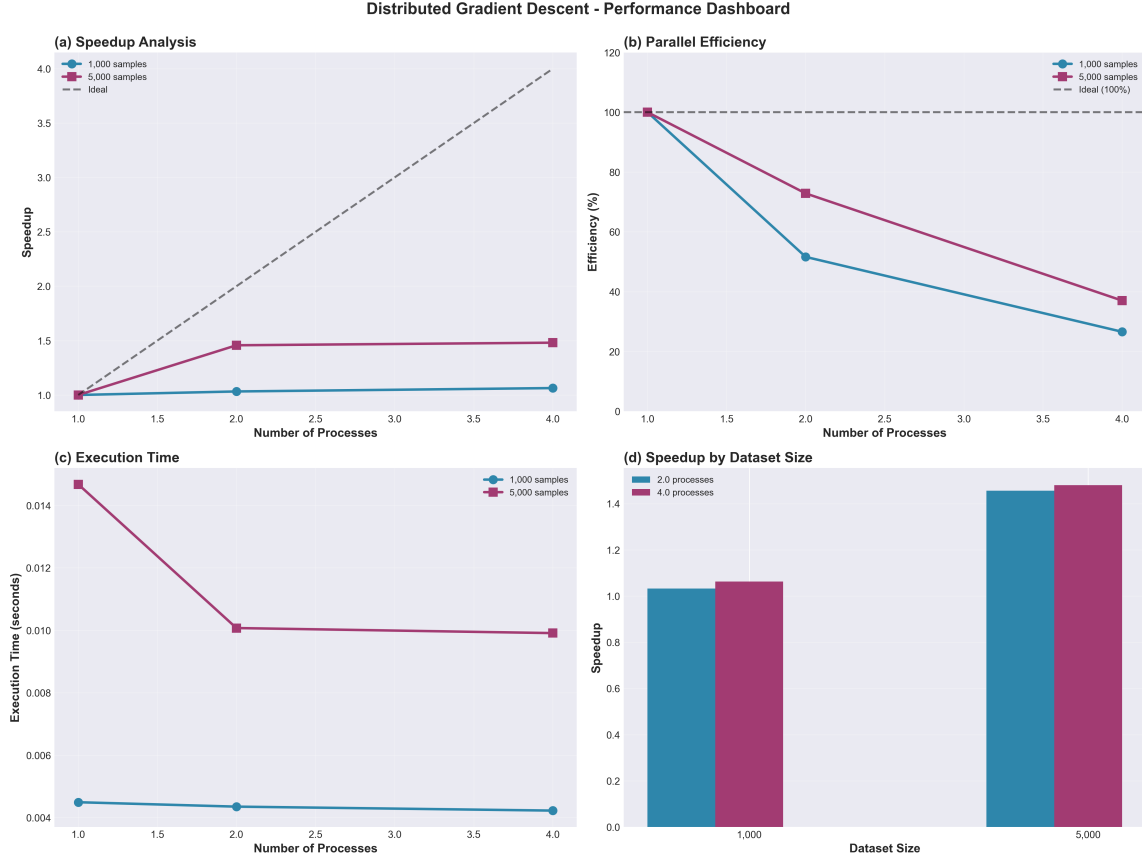


Figure 3: Performance dashboard showing speedup, efficiency, execution time, and comparative analysis for distributed gradient descent

Figure 3 presents a comprehensive performance analysis with four key visualizations:

(a) Speedup Analysis The speedup curves show:

- For 1,000 samples (blue): Speedup reaches approximately 1.05x with 4 processes, showing minimal improvement. This indicates that the dataset is too small to benefit from parallelization due to communication overhead.
- For 5,000 samples (magenta): Speedup improves to approximately 1.5x with 4 processes, demonstrating better scalability with larger datasets.
- Both curves remain well below the ideal linear speedup (dashed black line), indicating significant parallel overhead.

(b) Parallel Efficiency Efficiency analysis reveals:

- 1,000 samples: Efficiency drops to 26% with 4 processes, meaning only 26% of the parallel resources are effectively utilized.
- 5,000 samples: Efficiency reaches 37% with 4 processes, showing improved resource utilization with larger datasets.

- The steep decline from 100% (1 process) indicates that communication costs dominate with small workloads per process.

(c) Execution Time The execution time plots show:

- 1,000 samples: Execution time remains nearly constant (0.004s) regardless of the number of processes, suggesting overhead neutralizes parallelization benefits.
- 5,000 samples: Clear reduction in execution time from 0.015s (1 process) to 0.010s (4 processes), demonstrating effective parallelization for larger workloads.

(d) Speedup by Dataset Size The bar chart comparison reveals:

- Larger datasets benefit more from parallelization than smaller ones.
- The speedup gap between 2 and 4 processes is more pronounced for 5,000 samples, indicating better scalability.